

23CSE113

User Interface Design

Java Script

Java Script

- JavaScript is the world's most popular programming language.
- It is mainly used for **making web pages interactive**
- JavaScript is a **lightweight, interpreted, and object-oriented** programming language.
- Runs in the browser.
- Created in 1995 by **Brendan Eich** at **Netscape**

Java vs JavaScript???

- In 1995, a programmer named **Brendan Eich** created the language while working at **Netscape** (a major web browser company back then).
- Initially, it was called **Mocha**. Then, it was renamed **LiveScript** during development.
- Around that time, **Java** (a different programming language developed by Sun Microsystems) was very popular and "hot" in the tech world.
- **Netscape** decided to rename **LiveScript** to **JavaScript** to attract attention and benefit from **Java's** popularity — even though **Java** and **JavaScript** are completely different languages!
- **Java** is a full-fledged, compiled, object-oriented language.
- **JavaScript** is a lightweight, interpreted scripting language mostly used inside browsers.
- The name "JavaScript" was mainly a marketing strategy to make the new language more popular quickly.

Why JavaScript?

- **Client-Side Validation:** Checking user input before sending data to server.
- **Dynamic Web Pages:** Update page content without reloading the page.
- **Rich Interfaces:** Drag & drop, sliders, interactive elements.
- **Supported Everywhere:** Runs on all modern browsers and platforms.

Features of JavaScript

- **Lightweight:** Minimal syntax; easy to learn and implement.
- **Interpreted Language:** No compilation needed; browser interprets directly.
- **Object-Oriented:** Supports objects, inheritance, and encapsulation.
- **Event-Driven:** Reacts to user interactions like clicks, mouse movements.
- **Platform Independent:** Works across Windows, Mac, Linux, etc.

HTML vs. CSS vs. JavaScript

If you are familiar with JavaScript, you would know the relationship between HTML, CSS and JavaScript. Let's have a look at an example to understand the analogy.



- ❑ HTML(Hyper Text Markup Language) is more like the skeleton of the web. It is used for displaying the web.
- ❑ On the other hand, CSS is like our clothes. It makes the web look better. It uses CSS which stands for Cascading Style Sheets for styling purpose.
- ❑ Finally, JavaScript is used to add life to a web page. Just like how kids move around using the skateboard, the web also motions with the help of JavaScript.

What can JavaScript do?

JavaScript can change HTML content

JavaScript can change HTML attributes

JavaScript can change CSS style

JavaScript can hide HTML elements

JavaScript can show hidden HTML elements

Where To write JavaScript

- In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.
- You can place any number of scripts in an HTML document.
- Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript in Body</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>

</body>
</html>
```

JavaScript in Body

My First JavaScript

[W3Schools Tryit Editor](#)

Where to Insert JavaScript

JavaScript in <head>

JavaScript in <body>

JavaScript in an external file

JavaScript in an external url

JavaScript in an external folder

JavaScript in <head>

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</but

</body>
</html>
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_where_to_head

Demo JavaScript in Head

A Paragraph.

Try it

Demo JavaScript in Head

Paragraph changed.

Try it

In this example, a JavaScript function is placed in the <head> section of an HTML page.

The function is invoked (called) when a button is clicked:

JavaScript Functions and Events

- A JavaScript **function** is a block of JavaScript code, that can be executed when "called" for.
- For example, a function can be called when an **event** occurs, like when the user clicks a button

JavaScript in <body>

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A Paragraph</p>

<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

[W3Schools Tryit Editor](#)

Demo Javascript in Body

A Paragraph

Try it

Demo Javascript in Body

Paragraph changed.

Try it

In this example, a JavaScript function is placed in the <body> section of an HTML page.

The function is invoked (called) when a button is clicked:

External JavaScript

Scripts can also be placed in external files:

External file: myScript.js

```
function myFunction() {  
    document.getElementById("demo").innerHTML = "Paragraph changed.";  
}
```

- External scripts are practical when the same code is used in many different web pages.
 - JavaScript files have the file extension **.js**.
 - To use an external script, put the name of the script file in the **src** (source) attribute of a **<script>** tag:
-

Example

```
<script src="myScript.js"></script>
```

- You can place an external script reference in **<head>** or **<body>** as you like.
- The script will behave as if it was located exactly where the **<script>** tag is located.
- External scripts cannot contain **<script>** tags.

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo External JavaScript</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try
it</button>

<p>This example links to "myScript.js".</p>
<p>(myFunction is stored in "myScript.js")</p>

<script src="myScript.js"></script>

</body>
</html>
```

Demo External JavaScript

A Paragraph.

Try it

This example links to "myScript.js".

(myFunction is stored in "myScript.js")

[W3Schools Tryit Editor](#)

External References

- An external script can be referenced in 3 different ways:

 - With a full URL (a full web address)
 - With a file path (like /js/)
 - Without any path

This example uses a **full URL** to link to myScript.js:

```
<script src="https://www.w3schools.com/js/myScript.js"></script>
```

- This example uses a **file path** to link to myScript.js:

```
<script src="/js/myScript.js"></script>
```

This example uses no path to link to myScript.js:

```
<script src="myScript.js"></script>
```

External JavaScript Advantages

- Placing scripts in external files has some advantages:
- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

Class work

Create a webpage with:

- A heading
- A button
- When the button is clicked, **change the background color of the page**

Solution

`</>` HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Background Color Change</title>
</head>
<body>

<h3>Click the button to change background color</h3>

<button onclick="changeColor()">Change Color</button>

<script>
function changeColor() {
  document.body.style.backgroundColor = "lightblue";
}
</script>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<h2 id="box">Welcome Students</h2>

<button onclick="changeBg()">Change Background</button>

<script>
function changeBg() {
  document.getElementById("box").style.backgroundColor = "yellow";
}
</script>

</body>
</html>
```

JavaScript Output

- JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML` or `innerText`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

Using innerHTML

- To access an HTML element, you can use the `document.getElementById(id)` method.
- Use the `id` attribute to identify the HTML element.
- Then use the `innerHTML` property to change the HTML content of the HTML element:

```
<!DOCTYPE html>
<html>
<body>

<h1>My Web Page</h1>
```

```
<p id="demo"></p>
```

```
<script>
document.getElementById("demo").innerHTML = "<h2>Hello World</h2>";
</script>
```

```
</body>
</html>
```

My Web Page

Hello World

Using document.write()

- For testing purposes, it is convenient to use `document.write()`:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>My First Web Page</h2>
<p>My first paragraph.</p>
```

```
<p>Never call document.write after the document has finished loading.
It will overwrite the whole document.</p>
```

```
<script>
document.write(5 + 6);
</script>
```

```
</body>
</html>
```

My First Web Page

My first paragraph.

Never call document.write after the document has finished loading. It will overwrite the whole document.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>My First Web Page</h2>
```

```
<p>My first paragraph.</p>
```

```
<button type="button" onclick="document.write(5 + 6)">Try it</button>
```

```
</body>
```

```
</html>
```

My First Web Page

My first paragraph.

Try it

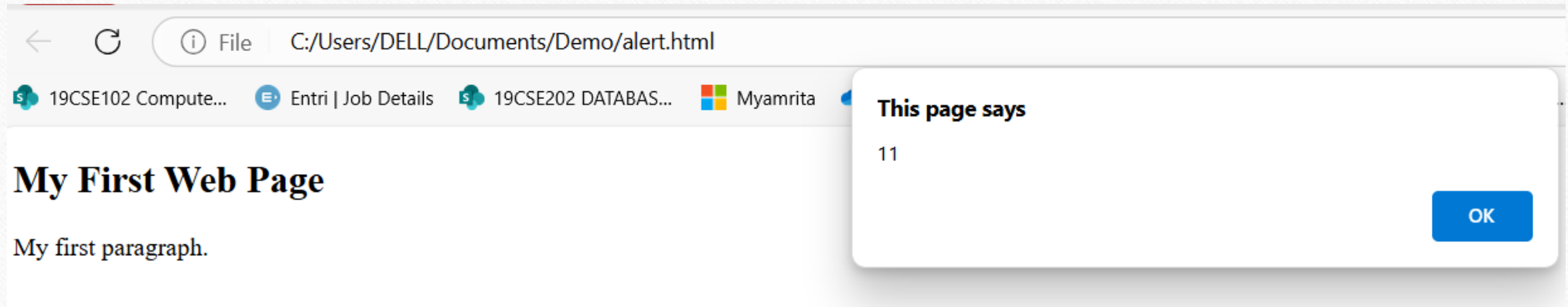
Using window.alert()

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html>
```



JavaScript Statements

JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments.

Example

- `let x, y, z;   // Statement 1`
- `x = 5;        // Statement 2`
- `y = 6;        // Statement 3`
- `z = x + y;     // Statement 4`

JavaScript Programs

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called **statements**.

-
- A **JavaScript program** is a list of programming **statements**.
 - In HTML, JavaScript programs are executed by the web browser.
 - Most JavaScript programs contain many JavaScript statements.
 - The statements are executed, one by one, in the same order as they are written.

JavaScript Code Blocks

- JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.
- The purpose of code blocks is to define statements to be executed together.
- One place you will find statements grouped together in blocks, is in JavaScript functions:

Example

```
function myFunction() {  
    document.getElementById("demo1").innerHTML = "Hello Dolly!";  
    document.getElementById("demo2").innerHTML = "How are you?";  
}
```

JavaScript Keywords

- JavaScript statements often start with a **keyword** to identify the JavaScript action to be performed.
- Our Reserved Words Reference lists all JavaScript keywords.
- JavaScript keywords are reserved words. Reserved words cannot be used as names for variables.

Here is a list of some of the keywords you will learn about in this tutorial:

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

Exercise[?]

How many statements can you find in this line of code

```
let a = 5; let b = 6; c = a + b;
```

JavaScript Variables

- Variables are Containers for Storing Data
- JavaScript Variables can be declared in 4 ways:
- Automatically
- Using **var**
- Using **let**
- Using **const**

- In this example, **x**, **y**, and **z** are undeclared variables.
- They are automatically declared when first used:

JavaScript Variables

In this example, x, y, and z are undeclared.

They are automatically declared when first used.

The value of z is: 11

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are undeclared.</p>
<p>They are automatically declared when first used.</p>

<p id="demo"></p>

<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, price1, price2, and total are variables.

The total is: 11

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Variables</h1>
```

```
<p>In this example, price1, price2, and total are variables.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const price1 = 5;
```

```
const price2 = 6;
```

```
let total = price1 + price2;
```

```
document.getElementById("demo").innerHTML
```

```
"The total is: " + total;
```

```
</script>
```

```
</body>
```

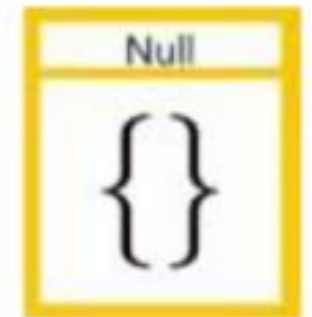
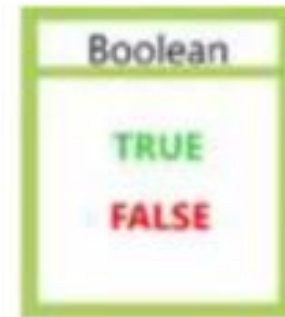
```
</html>
```

- The two variables price1 and price2 are declared with the const keyword.
- These are constant values and cannot be changed.
- The variable total is declared with the let keyword.
- The value total can be changed.

Data Types

You can assign different types of values to a variable such as a number or a string. There are different data types such as:

- Numbers
- Strings
- Boolean
- Undefined
- Null



The Concept of Data Types

```
let length = 16;           // Number
let lastName = "Johnson";  // String
let x = {firstName:"John", lastName:"Doe"};  // Object
```

```
let x = 16 + "Volvo";
```

Result: 16Volvo

```
let x = "16" + "Volvo"
```

Result: 16Volvo

When adding a number and a string, JavaScript will treat the number as a string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript</h2>

<p>When adding a number and a string, JavaScript will treat
the number as a string.</p>

<p id="demo"></p>

<script>
let x = 16 + "Volvo";
document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Result:

JavaScript

When adding a number and a string, JavaScript will treat the number as a string.

16Volvo

JavaScript evaluates expressions from left to right. Different sequences can produce different results:

```
let x = 16 + 4 + "Volvo";
```

Result: 20Volvo

JavaScript treats 16 and 4 as numbers, until it reaches "Volvo".

```
let x = "Volvo" + 16 + 4;
```

Result: Volvo164

Since the first operand is a string, all operands are treated as strings.

JavaScript Types are Dynamic

JavaScript has dynamic types. This means that the same variable can be used to hold different data types:

```
let x;           // Now x is undefined  
x = 5;           // Now x is a Number  
x = "John";      // Now x is a String
```

var is function-scoped:

javascript

```
function testVar() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); //  10 (accessible outside the block)  
}
```

let is block-scoped:

javascript

```
function testLet() {  
  if (true) {  
    let y = 20;  
  }  
  console.log(y); //  ReferenceError  
}
```



Arrays and Objects

- Refer
- Arrays
- Objects

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators

JavaScript Operators

Assignment:

The assignment operator (=) assigns a value to a variable.

```
let x = 5;           // assign the value 5 to x
let y = 2;           // assign the value 2 to y
let z = x + y;       // assign the value 7 to z (5 + 2)
```

Adding:

The addition operator (+) adds numbers:

```
let x = 5;
let y = 2;
let z = x + y;
```

Multiplying

The multiplication operator (*) multiplies numbers.

```
let x = 5;  
let y = 2;  
let z = x * y;
```

JavaScript Arithmetic Operators are used to perform arithmetic on numbers:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

JavaScript Assignment Operators

Assignment operators assign values to JavaScript variables.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

The addition assignment operator (+=) adds a value to a variable.

```
let x = 10;
```

```
x += 5;           // x will be 10+5=15
```

JavaScript String Operators

The + operator can also be used to add (concatenate) strings.

```
let text1 = "John";  
let text2 = "Doe";  
let text3 = text1 + " " + text2;
```

The result of text3 will be: John Doe

The += assignment operator can also be used to add (concatenate) strings:

```
let text1 = "What a very ";  
text1 += "nice day";
```

The result of text1 will be: What a very nice day

JavaScript Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JavaScript Logical Operators

Operator	Description
&&	logical and
	logical or
!	logical not

JavaScript Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

JavaScript if, else, and else if

- Conditional statements are used to perform different actions based on different conditions.
- In JavaScript we have the following conditional statements:
 - Use **if** to specify a block of code to be executed, if a specified condition is true
 - Use **else** to specify a block of code to be executed, if the same condition is false
 - Use **else if** to specify a new condition to test, if the first condition is false
 - Use **switch** to specify many alternative blocks of code to be executed

The if Statement

Use the `if` statement to specify a block of JavaScript code to be executed if a condition is true.

Syntax

Note that `if` is in lowercase letters. Uppercase letters (If or IF) will generate a JavaScript error.

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

The else Statement

Use the `else` statement to specify a block of code to be executed if the condition is false.

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

The else if Statement

Use the `else if` statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript if .. else</h2>
<p>A time-based greeting:</p>
<p id="demo"></p>
```

```
<script>
const time = new Date().getHours();
let greeting;
if (time < 10) {
    greeting = "Good morning";
} else if (time < 20) {
    greeting = "Good day";
} else {
    greeting = "Good evening";
}
document.getElementById("demo").innerHTML = greeting;
</script>

</body>
</html>
```

This code gets the **current hour** of the day (based on your computer's local time) using JavaScript's Date object.

JavaScript if .. else

A time-based greeting:

Good morning

Homework 1: Simple `if`

- **Task:** Show a message if a user is old enough to drive.

```
let age = 20;
```

```
if (age >= 18) {
```

```
  // Show: "You can drive"
```

```
}
```

Your task: Replace the comment with code that displays "You can drive" on the webpage using `innerHTML`.

The JavaScript Switch Statement

Use the `switch` statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

```
<p id="demo"></p>
<script>
let day;
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
    day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case 6:
    day = "Saturday";
  }
document.getElementById("demo").innerHTML = "Today is " + day;
</script>
```

The `getDay()` method returns the weekday as a number between 0 and 6. (Sunday=0, Monday=1, Tuesday=2 ..)

This example uses the weekday number to calculate the weekday name:

JavaScript For Loop

- Loops can execute a block of code a number of times.
- Loops are handy, if you want to run the same code over and over again, each time with a different value.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript For Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const cars = ["BMW", "Volvo", "Saab", "Ford", "Fiat", "Audi"];
```

```
let text = "";
```

```
for (let i = 0; i < cars.length; i++) {  
  text += cars[i] + "<br>";  
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript For Loop

BMW

Volvo

Saab

Ford

Fiat

Audi

Different Kinds of Loops

JavaScript supports different kinds of loops:

- `for` - loops through a block of code a number of times
- `for/in` - loops through the properties of an object
- `for/of` - loops through the values of an iterable object
- `while` - loops through a block of code while a specified condition is true
- `do/while` - also loops through a block of code while a specified condition is true

Refer

- **JavaScript For In**
- **JavaScript For Of**
- **JavaScript While Loop**

-
- Write a program to display “Hello, World!” using JavaScript.
 - Write a JavaScript program to swap two numbers.

JavaScript Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).
- A JavaScript function is defined with the **function** keyword, followed by a **name**, followed by parentheses ().
- Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- The parentheses may include parameter names separated by commas:
(parameter1, parameter2, ...).
- The code to be executed, by the function, is placed inside curly brackets: {}.

-
- Function **parameters** are listed inside the parentheses () in the function definition.
 - Function **arguments** are the **values** received by the function when it is invoked.
 - Inside the function, the arguments (the parameters) behave as local variables.

Function Invocation

The code inside the function will execute when "something" **invokes** (calls) the function:

- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code
- Automatically (self invoked)

Function Return

- When JavaScript reaches a **return** statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a **return value**.
- The return value is "returned" back to the "caller"

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
```

```
<p>Call a function which performs a calculation and returns the result:</p>
```

```
<p id="demo"></p>
```

```
<script>
function myFunction(p1, p2) {
  return p1 * p2;
}
```

```
let result = myFunction(4, 3);
document.getElementById("demo").innerHTML = result;
</script>
```

```
</body>
</html>
```

JavaScript Functions

Call a function which performs a calculation and returns the result:

12

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>
let x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {
  return a * b;
}
</script>
</body>
</html>
```

JavaScript Functions

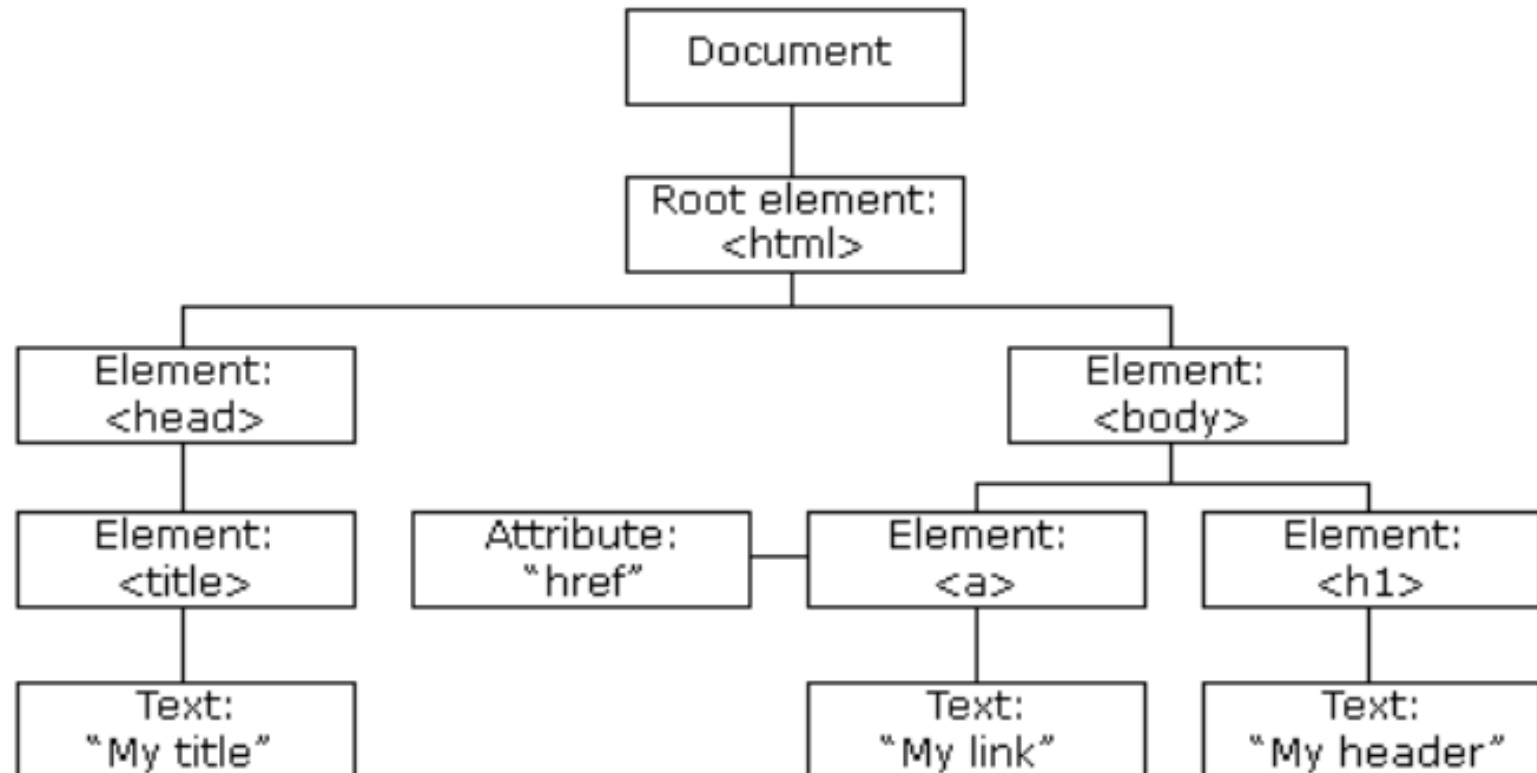
Call a function which performs a calculation and returns the result:

12

The HTML DOM (Document Object Model)

- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- The **HTML DOM** model is constructed as a tree of **Objects**:

The HTML DOM Tree of Objects



Example HTML Page

</> HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
</body>
</html>
```



DOM Tree Structure

When the browser loads this, the Document Object Model becomes:

```
Document
├── html
│   ├── head
│   │   └── title
│   │       └── "My Page"
│   └── body
│       ├── h1
│       │   └── "Welcome"
│       └── p
│           └── "This is a paragraph"
```



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

We will learn..

- How to change the content of HTML elements
- How to change the style (CSS) of HTML elements
- How to react to HTML DOM events
- How to add and delete HTML elements

What is the DOM?

- The DOM is a W3C (World Wide Web Consortium) standard.
- The DOM defines a standard for accessing documents:
- *The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document.*
- The W3C DOM standard is separated into 3 different parts:
 - Core DOM - standard model for all document types
 - XML DOM - standard model for XML documents
 - HTML DOM - standard model for HTML documents

What is the HTML DOM?

- The HTML DOM is a standard **object** model and **programming interface** for HTML.

It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements
- In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

What Does the HTML DOM Look Like?

Imagine your webpage as a tree

- The document is the root.
- HTML tags like `<html>`, `<head>`, and `<body>` are branches.
- Attributes, text, and other elements are the leaves.

JavaScript - HTML DOM Methods

- HTML DOM methods are **actions** you can perform (on HTML Elements).
- HTML DOM properties are **values** (of HTML Elements) that you can set or change.

The DOM Programming Interface

- The HTML DOM can be accessed with JavaScript (and with other programming languages).
- In the DOM, all HTML elements are defined as **objects**.
- The programming interface is the properties and methods of each object.
- A **property** is a value that you can get or set (like changing the content of an HTML element).
- A **method** is an action you can do (like add or deleting an HTML element).

The following example changes the content (the **innerHTML**) of the `<p>` element with `id="demo"`:

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Page</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

In the example
, `getElementById` is
a **method**,
while `innerHTML` is
a **property**.

My First Page
Hello World!

The getElementById Method

- The most common way to access an HTML element is to use the **id** of the element.
- In the example above the **getElementById** method used **id="demo"** to find the element.

The innerHTML Property

- The easiest way to get the content of an element is by using the **innerHTML** property.
- The **innerHTML** property is useful for getting or replacing the content of HTML elements.
- The **innerHTML** property can be used to get or change any HTML element, including **<html>** and **<body>**.

The HTML DOM Document Object

- The document object represents your web page.
- If you want to access any element in an HTML page, you always start with accessing the document object.
- Below are some examples of how you can use the document object to access and manipulate HTML.

Finding HTML Elements

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Changing HTML Elements

Property	Description
<code>element.innerHTML = new html content</code>	Change the inner HTML of an element
<code>element.attribute = new value</code>	Change the attribute value of an HTML element
<code>element.style.property = new style</code>	Change the style of an HTML element
Method	Description
<code>element.setAttribute(attribute, value)</code>	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Adding Events Handlers

Method	Description
<code>document.getElementById(<i>id</i>).onclick = function(){<i>code</i>}</code>	Adding event handler code to an onclick event

Finding HTML Objects

The first HTML DOM Level 1 (1998), defined 11 HTML objects, object collections, and properties. These are still valid in HTML5.

Later, in HTML DOM Level 3, more objects, collections, and properties were added.

Property	Description	DOM
document.anchors	Deprecated. Do not use it.	1
document.applets	Deprecated. Do not use it.	1
document.baseURI	Returns the absolute base URI of the document	3
document.body	Returns the <body> element	1
document.cookie	Returns the document's cookie	1
document.doctype	Returns the document's doctype	3
document.documentElement	Returns the <html> element	3
document.documentMode	Returns the mode used by the browser	3
document.documentURI	Returns the URI of the document	3
document.domain	Deprecated. Do not use it.	1

document.embeds	Returns all <embed> elements	3
document.forms	Returns all <form> elements	1
document.head	Returns the <head> element	3
document.images	Returns all elements	1
document.implementation	Returns the DOM implementation	3
document.inputEncoding	Returns the document's encoding (character set)	3
document.lastModified	Returns the date and time the document was updated	3
document.links	Returns all <area> and <a> elements that have a href attribute	1
document.readyState	Returns the (loading) status of the document	3
document.referrer	Returns the URI of the referrer (the linking document)	1
document.scripts	Returns all <script> elements	3
document.strictErrorChecking	Returns if error checking is enforced	3
document.title	Returns the <title> element	1
document.URL	Returns the complete URL of the document	1

JavaScript HTML DOM Elements

[< Previous](#)

Finding HTML Elements

- With JavaScript, you want to manipulate HTML elements.
- To do so, you have to find the elements first. There are several ways to do this:
- Finding HTML elements by id
- Finding HTML elements by tag name
- Finding HTML elements by class name
- Finding HTML elements by CSS selectors
- Finding HTML elements by HTML object collections

Finding HTML Element by Id

The easiest way to find an HTML element in the DOM, is by using the element id.

This example finds the element with `id="intro"` :

Example

```
const element = document.getElementById("intro");
```

If the element is found, the method will return the element as an object (in element).

If the element is not found, element will contain `null`.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript HTML DOM</h2>
```

```
<p id="intro">Finding HTML Elements by Id</p>
```

```
<p>This example demonstrates the <b>getElementsById</b> method.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const element = document.getElementById("intro");
```

```
document.getElementById("demo").innerHTML =
```

```
"The text from the intro paragraph is: " + element.innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript HTML DOM

Finding HTML Elements by Id

This example demonstrates the **getElementsById** method.

The text from the intro paragraph is: Finding HTML Elements by Id

Finding HTML Elements by Tag Name

This example finds all `<p>` elements:

Example

```
const element = document.getElementsByTagName("p");
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript HTML DOM</h2>
```

```
<p>Finding HTML Elements by Tag Name.</p>
```

```
<p>This example demonstrates the <b>getElementsByTagName</b> method.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const element = document.getElementsByTagName("p");
```

```
document.getElementById("demo").innerHTML = 'The text in first paragraph (index 0) is: '  
+ element[0].innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript HTML DOM

Finding HTML Elements by Tag Name.

This example demonstrates the **getElementsByTagName** method.

The text in first paragraph (index 0) is: Finding HTML Elements by Tag Name.

Predict the O/P

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<div id="main">
<p>Finding HTML Elements by Tag Name</p>
<p>This example demonstrates the <b>getElementsByTagName</b> method.</p>
</div>

<p id="demo"></p>

<script>
const x = document.getElementById("main");
const y = x.getElementsByTagName("p");

document.getElementById("demo").innerHTML =
'The first paragraph (index 0) inside "main" is: ' + y[0].innerHTML;

</script>

</body>
</html>
```

This example finds the element with `id="main"`, and then finds all `<p>` elements inside "main":

JavaScript HTML DOM

Finding HTML Elements by Tag Name

This example demonstrates the **getElementsByTagName** method.

The first paragraph (index 0) inside "main" is: Finding HTML Elements by Tag Name

Finding HTML Elements by Class Name

If you want to find all HTML elements with the same class name, use `getElementsByClassName()`

This example returns a list of all elements with `class="intro"`.

Example

```
const x = document.getElementsByClassName("intro");
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript HTML DOM</h2>
```

```
<p>Finding HTML Elements by Class Name.</p>
```

```
<p class="intro">Hello World!</p>
```

```
<p class="intro">This example demonstrates the <b>getElementsByClassName</b> method</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const x = document.getElementsByClassName("intro");
```

```
document.getElementById("demo").innerHTML =
```

```
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript HTML DOM

Finding HTML Elements by Class Name.

Hello World!

This example demonstrates the **getElementsByClassName** method.

The first paragraph (index 0) with class="intro" is: Hello World!

Finding HTML Elements by CSS Selectors

If you want to find all HTML elements that match a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.

This example returns a list of all `<p>` elements with `class="intro"`.

Example

```
const x = document.querySelectorAll("p.intro");
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript HTML DOM</h2>
```

```
<p>Finding HTML Elements by Query Selector</p>
```

```
<p class="intro">Hello World!.</p>
```

```
<p class="intro">This example demonstrates the <b>querySelectorAll</b> method.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const x = document.querySelectorAll("p.intro");
```

```
document.getElementById("demo").innerHTML =
```

```
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;
```

```
</script>
```

```
</body>
```

JavaScript HTML DOM

Finding HTML Elements by Query Selector

Hello World!.

This example demonstrates the **querySelectorAll** method.

The first paragraph (index 0) with class="intro" is: Hello World!.

Changing the Value of an Attribute

- To change the value of an HTML attribute, use this syntax:
- `document.getElementById(id).attribute = new value`
- This example changes the value of the src attribute of an `<img>` element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>


<script>
document.getElementById("image").src = "landscape.jpg";
</script>

<p>The original image was smiley.gif, but the script changed it to landscape.jpg</p>

</body>
</html>
```

The original image was smiley.gif, but the script changed it to landscape.jpg

-
- The HTML document above contains an `<img>` element with `id="myImage"`
 - We use the HTML DOM to get the element with `id="myImage"`
 - A JavaScript changes the `src` attribute of that element from "smiley.gif" to "landscape.jpg"

- The following HTML objects (and object collections) are also accessible:

- [document.anchors](#)
- [document.body](#)
- [document.documentElement](#)

- [document.embeds](#)
- [document.forms](#)
- [document.head](#)
- [document.images](#)
- [document.links](#)
- [document.scripts](#)
- [document.title](#)

Form Validation

- JavaScript form validation checks user input before submitting the form to ensure it's correct.
- It helps catch errors and improves the user experience.

JavaScript Form Validation

- HTML form validation can be done by JavaScript.
- If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

JavaScript Example

```
function validateForm() {  
    let x = document.forms["myForm"]["fname"].value;  
    if (x == "") {  
        alert("Name must be filled out");  
        return false;  
    }  
}
```

(Find the input field named **fname** inside the form named **myForm**, get what the user typed, and store it in the variable **x**.)

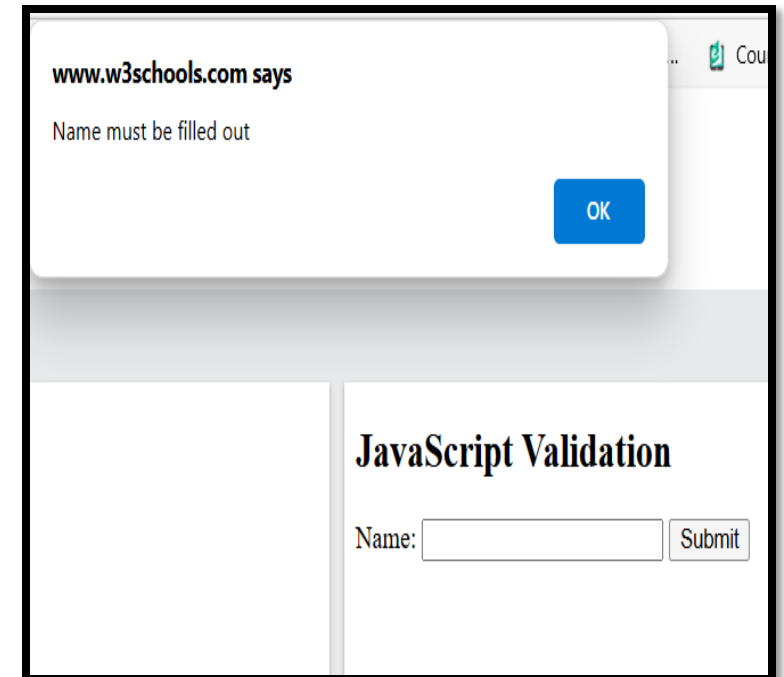
```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
    let x = document.forms["myForm"]["fname"].value;
    if (x == "") {
        alert("Name must be filled out");
        return false;
    }
    return true; // Always include this
}

</script>
</head>
<body>

<h2>JavaScript Validation</h2>

<form name="myForm" action="/action_page.php" onsubmit="return
validateForm()" method="post">
    Name: <input type="text" name="fname">
    <input type="submit" value="Submit">
</form>

</body>
</html>
```



What Actually happens?

- User clicks “Submit.”
- `onsubmit="return validateForm()"` is triggered.
 - The browser calls the JavaScript function `validateForm()` before sending the form data.
- Inside the `validateForm()` function:
 - `let x = document.forms["myForm"]["fname"].value;` (It grabs the value typed in the text box (fname).)
- `if (x == "") {`
- `alert("Name must be filled out");`
- `return false;`
- `}`
- If the user didn't type anything, it shows an alert:
“Name must be filled out”,
and then return false → **blocks the form from submitting.**

When an input entered

Submitted Form Data

Your input was received as:

```
fname=anju sukumar
```

The server has processed your input and returned this answer.

This code is written in
`action_page.php`

JavaScript Can Validate Numeric Input

```
<html>
<body>

<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>

<input type="text" id="numb">

<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>
```

JavaScript Validation

Please input a number between 1 and 10

Input OK

Automatic HTML Form Validation

- HTML form validation can be performed automatically by the browser:
- If a form field (fname) is empty, the **required** attribute prevents this form from being submitted:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript Validation</h2>
```

```
<form action="/action_page.php" method="post">
  <input type="text" name="fname" required>
  <input type="submit" value="Submit">
</form>
```

```
<p>If you click submit, without filling out the text field,
your browser will display an error message.</p>
```

```
</body>
</html>
```

JavaScript Validation

! Please fill out this field.

filling out the text field, your browser will display an error m

-
- Write a JavaScript program to validate that the user's age is greater than 18.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<script>
```

```
function validateAge() {  
  let age =  
    document.getElementById("age").value;  
  if (isNaN(age) || age <= 18) {  
    alert("You must be at least 18 years  
old");  
    return false;  
  }  
  return true; }
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<h2>Age Validation Form</h2>
```

```
<form onsubmit="return validateAge()">  
  Enter your age: <input type="text"  
id="age"><br><br>  
  <input type="submit" value="Submit">  
</form>
```

```
</body>
```

```
</html>
```

`isNaN(age)` means:

- "Is the value of **age** **Not a Number**?"
- In JavaScript, `isNaN( )` is a built-in function that checks whether a value is **NaN (Not-a-Number)**. It returns:
- `true` → if `age` is not a number
- `false` → if `age` is a number

JavaScript HTML DOM Events

- HTML DOM allows JavaScript to react to HTML events:
- Common DOM events: click, submit, mouseover, keydown, load, etc.

Reacting to Events

- A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:
 - ***onclick=JavaScript***
- Examples of HTML events:
 - When a user clicks the mouse
 - When a web page has loaded
 - When an image has been loaded
 - When the mouse moves over an element
 - When an input field is changed
 - When an HTML form is submitted
 - When a user strokes a key

In this example, the content of the `<h2>` element is changed when a user clicks on it:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onclick Attribute</h2>
```

JavaScript HTML Events

The onclick Attribute

Click on this text!

```
<h2 onclick="this.innerHTML='Oops!'">Click on this text!</h2>
```

```
</body>
</html>
```

JavaScript HTML Events

The onclick Attribute

Oops!

-
- `this` refers to **the element itself** (`<h2>` in this case).
 - `this.innerHTML = 'Ooops!'` means:
 - When this heading (`<h2>`) is clicked, change its inner text (HTML content) to `'Ooops!'`.

In this example, a function is called from the event handler:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onclick Attribute</h2>
```

```
<h2 onclick="changeText(this)">Click on this text!</h2>
```

```
<script>
function changeText(id) {
    id.innerHTML = "Ooops!";
}
</script>
```

```
</body>
</html>
```

JavaScript HTML Events

The onclick Attribute

Click on this text!

JavaScript HTML Events

The onclick Attribute

Ooops!

-
- An **event handler** is a **function or block of code** that runs **in response to a specific event** (like a click, keypress, form submission, etc.) happening on a web page.
 -  Think of it like this:
 -  An **event** is the bell (like a click).
 -  The **event handler** is the person who responds when the bell rings.

HTML Event Attributes

In the example above, a function named `displayDate` will be executed when the button is clicked.

JavaScript HTML Events

The onclick Attribute

Click the button to display the date.

The time is?

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onclick Attribute</h2>
```

```
<p>Click the button to display the date.</p>
<button onclick="displayDate()">The time is?</button>
```

```
<script>
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>
```

```
<p id="demo"></p>
```

```
</body>
</html>
```

To assign events to HTML elements you can use event attributes.

JavaScript HTML Events

The onclick Attribute

Click the button to display the date.

The time is?

Tue May 27 2025 12:12:37 GMT+0530 (India Standard Time)

Assigning onclick using JavaScript

html

```
<button id="myButton">The time is?</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("myButton").onclick = function() {
```

```
    document.getElementById("demo").innerHTML = Date();
```

```
};
```

```
</script>
```

onclick = function()

Assigns a **function to run later**, when the click event happens

Assign Events Using the HTML DOM

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onclick Events</h2>

<p>Click "Try it" to execute the displayDate() function.</p>
<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").onclick = displayDate;

function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

In the example above, a function named displayDate is assigned to an HTML element with the id="myBtn".

The function will be executed when the button is clicked.

The onclick Events

Click "Try it" to execute the displayDate() function.

Try it

Tue May 27 2025 12:15:21 GMT+0530 (India Standard Time)

Syntax

```
onclick = displayDate
```

```
onclick = displayDate()
```

Meaning

Assigns the function to be called **later**

Calls the function **immediately** and assigns the result (usually `undefined`)

The onmouseover and onmouseout Events

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onmouseover Attribute</h2>

<div onmouseover="mOver(this)" onmouseout="mOut(this)"
style="background-color:#D94A38;width:120px;height:20px;padding:40px;">
Mouse Over Me</div>

<script>
function mOver(obj) {
    obj.innerHTML = "Thank You"
}

function mOut(obj) {
    obj.innerHTML = "Mouse Over Me"
}
</script>

</body>
</html>
```

JavaScript HTML Events

The onmouseover Attribute



Mouse Over Me

• More Examples

- onmousedown and onmouseup
Change an image when a user holds down the mouse button.
- onload
Display an alert box when the page has finished loading.
- onfocus
Change the background-color of an input field when it gets focus.
- Mouse Events
Change the color of an element when the cursor moves over it.

JavaScript HTML DOM EventListener

- The addEventListener() method

Example

Add an event listener that fires when a user clicks a button:

```
document.getElementById("myBtn").addEventListener("click", displayDate);
```

The `addEventListener()` method attaches an event handler to the specified element.

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element.

You can add many event handlers of the same type to one element, i.e two "click" events.

You can add event listeners to any DOM object not only HTML elements. i.e the window object.

The `addEventListener()` method makes it easier to control how the event reacts to bubbling.

When using the `addEventListener()` method, the JavaScript is separated from the HTML markup, for better readability and allow you to add event listeners even when you do not control the HTML markup.

You can easily remove an event listener by using the `removeEventListener()` method.

Syntax

```
element.addEventListener(event, function, useCapture);
```

The first parameter is the type of the event (like "click" or "mousedown" or any other HTML DOM Event.)

The second parameter is the function we want to call when the event occurs.

The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

Add an Event Handler to an Element

Example

Alert "Hello World!" when the user clicks on an element:

```
element.addEventListener("click", function(){ alert("Hello World!"); });
```

The removeEventListener() method

The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method:

Example

```
element.removeEventListener("mousemove", myFunction);
```